

12. Nonlinear equations and optimization

- nonlinear equations
- fixed point iteration
- Newton method for nonlinear equations
- unconstrained optimization
- gradient and Newton methods for optimization

Set of nonlinear equations

consider n nonlinear equations in n variables $x_1, x_2, \dots, x_n$:

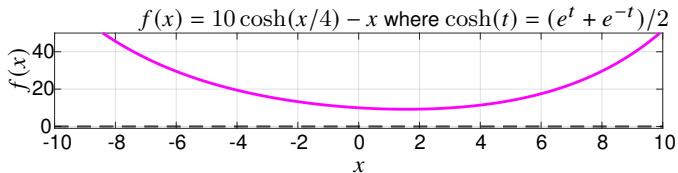
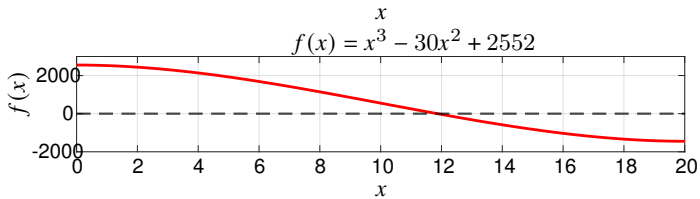
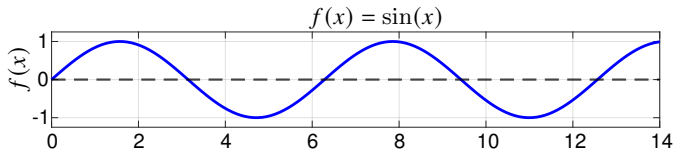
$$f_i(x_1, \dots, x_n) = 0, \quad i = 1, \dots, n$$

in vector notation: $f(x) = 0$ with

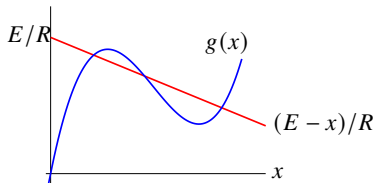
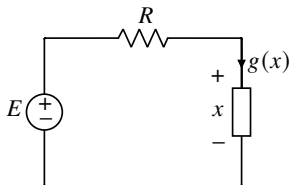
$$x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}, \quad f(x) = \begin{bmatrix} f_1(x_1, \dots, x_n) \\ f_2(x_1, \dots, x_n) \\ \vdots \\ f_n(x_1, \dots, x_n) \end{bmatrix}$$

- $f : \mathbb{R}^n \rightarrow \mathbb{R}^n$ and $f_i : \mathbb{R}^n \rightarrow \mathbb{R}$ (assumed to be continuous)
- $f_i(x)$ is i th *residual* and $f(x)$ is *residual vector*
- $x^\star$ is a *root* or *zero* or *solution* if $f(x^\star) = 0$
- there may be one solution, multiple solutions, or no solution

Examples in one variable



Example: nonlinear resistive circuit



$$g(x) - \frac{E - x}{R} = 0$$

a nonlinear equation in the variable x , with three solutions

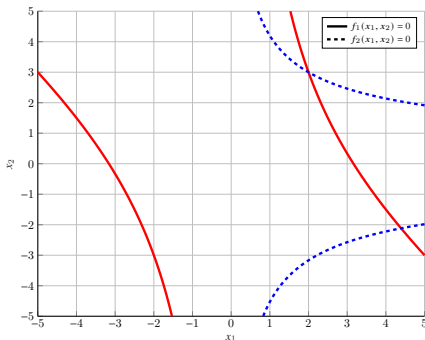
Example: nonlinear system

two nonlinear equations with two unknowns:

$$f_1(x_1, x_2) = x_1^2 + x_1x_2 - 10 = 0$$

$$f_2(x_1, x_2) = x_2 + 3x_1x_2^2 - 57 = 0$$

solution: values (x_1, x_2) such that both $f_1(x_1, x_2) = 0$ and $f_2(x_1, x_2) = 0$



one solution is $x_1 = 2, x_2 = 3$

Difficulty of solving nonlinear equations

- solving nonlinear equations is much harder than solving linear equations
- even determining if a solution exists is hard
- so we will use heuristic algorithms
- not guaranteed to always work
- but often work well in practice

Iterative methods

- nonlinear equations are much difficult to solve compared to linear equations
- obtaining a solution by finite-step algorithm is not feasible

Iterative algorithms

- start with *initial* or *starting point*, $x^{(1)}$ and compute estimates

$$x^{(1)}, x^{(2)}, \dots, x^{(k)}, \dots$$

- moving from $x^{(k)}$ to $x^{(k+1)}$ is called an *iteration* of the algorithm
- ideally converge to a root of the target function

$$x^{(k)} \rightarrow x^{\star} \quad \text{as} \quad k \rightarrow \infty$$

where $f(x^{\star}) = 0$

Convergence rates

- assume the sequence $x^{(k)}$ converges to a limit $x^\star$
- we assume scalar sequence, same concept hold for vector sequence

Linear convergence: if there exists a constant $c \in (0, 1)$ such that

$$|x^{(k)} - x^\star| \leq c |x^{(k-1)} - x^\star| \quad \text{for sufficiently large } k$$

example: $x^{(k)} = 1 + (1/2)^k$ linearly converges to $x^\star = 1$,

$$|x^{(k)} - x^\star| = (1/2)^k = \frac{1}{2} |x^{(k-1)} - x^\star|$$

satisfies the definition with $c = 1/2$

R-linear convergence: if a positive constant M and a value $c \in (0, 1)$ exist such that

$$|x^{(k)} - x^\star| \leq M c^k \quad \text{for sufficiently large } k$$

linear convergence implies R -linear convergence (reverse is not necessarily true)

Convergence rates

Superlinear convergence: if a sequence $c_k > 0$ with $c_k \rightarrow 0$ exists such that

$$|x^{(k)} - x^\star| \leq c_k |x^{(k-1)} - x^\star| \quad \text{for sufficiently large } k$$

example: $x^{(k)} = 1 + (1/(k+1))^k$ has superlinear convergence to $x^\star = 1$, as

$$|x^{(k)} - x^\star| = \frac{1}{(k+1)^k} = \frac{k^{k-1}}{(k+1)^k} \frac{1}{k^{k-1}} = \frac{k^{k-1}}{(k+1)^k} |x^{(k-1)} - x^\star|$$

satisfies the definition with $c_k = k^{k-1}/(k+1)^k$, which approaches zero

Quadratic convergence: if a constant $c > 0$ exists such that

$$|x^{(k)} - x^\star| \leq c |x^{(k-1)} - x^\star|^2 \quad \text{for sufficiently large } k$$

example: $x^{(k)} = 1 + (1/2)^{2^k}$ has quadratic convergence to $x^\star = 1$, as

$$|x^{(k)} - x^\star| = (1/2)^{2^k} = \left((1/2)^{2^{k-1}} \right)^2 = |x^{(k-1)} - x^\star|^2$$

satisfies the definition with $c = 1$

The bisection method

a method for finding root of a scalar function of a scalar $f : \mathbb{R} \rightarrow \mathbb{R}$

given: a, b with $a < b$, $f(a)f(b) < 0$, and tolerance ϵ

repeat

1. $x = (a + b)/2$
 2. compute $f(x)$; **if** $f(x) = 0$, **return** x
 3. **if** $f(x)f(a) < 0$, $b = x$, **else**, $a = x$
 4. **stop** if $b - a \leq \epsilon$
-

- condition $f(a)f(b) < 0$ ensures a root exists between a, b
- a, b can be chosen from graphing the function

Convergence

let $[a^{(k)}, b^{(k)}]$ be the interval after iteration k , then

$$b^{(k)} - a^{(k)} = (b - a)/2^k$$

- after k iterations, the midpoint $x^{(k)} = (b^{(k)} + a^{(k)})/2$ satisfies

$$|x^{(k)} - x^*| \leq b^{(k)} - a^{(k)} \leq (1/2)^k (b - a)$$

thus, it is R-linearly convergent with $c = 1/2$ and $M = b - a$

- the exit condition $b^{(k)} - a^{(k)} \leq \epsilon$ will be satisfied if

$$\log_2 \left(\frac{b - a}{2^k} \right) = \log_2(b - a) - k \leq \log_2 \epsilon$$

the algorithm therefore terminates after

$$k \approx \left\lceil \log_2 \left(\frac{b - a}{\epsilon} \right) \right\rceil$$

iterations where $\lceil \alpha \rceil$ is the smallest integer greater than or equal to α

MATLAB implementation

```
function [x,k] = bisect(f,a,b,tol)
% assuming func is a defined input function
% this function returns in p a value such that | x - root | < atol
% and in k the number of iterations required.
fa=f(a);fb=f(b);
if (a >= b) | (fa*fb >= 0) | (tol <= 0)
disp('something wrong with the input: quitting');
x = NaN; k=NaN;
return
end
k = ceil(log2(b-a) - log2(tol));
for i=1:k
x = (a+b)/2;
fx = f(x);
if abs(fx) < eps, k = i; return, end
if fa * fx < 0
b = x; fb = fx;
else
a = x; end
end
end
```

Examples

Example 1: $f(x) = x^3 - 30x^2 + 2552$

- initial interval $[0, 20]$ with a tolerance of 1×10^{-8}
- the associated MATLAB script is:

```
f = @(x) x^3 - 30*x^2 + 2552;  
[x,k] = bisect(f,0,20,1e-8)
```

the method converges to $x^* \approx 11.86150151$ after 31 iterations

Example 2: $f(x) = 2.5 \sinh(x/4) - 1$

- initial interval $[-10, 10]$ and tolerance of 1×10^{-10}
- the associated MATLAB script is:

```
f = @(x) 2.5 * sinh (x/4) - 1;  
[x,k] = bisect(f,-10,10,1e-10)
```

the method converges to $x^* \approx 1.5601412791$ after 38 iterations

Outline

- nonlinear equations
- **fixed point iteration**
- Newton method for nonlinear equations
- unconstrained optimization
- gradient and Newton methods for optimization

Fixed-point iteration for one variable

rearrange $f(x) = 0$ into *fixed-point* form

$$x = g(x)$$

- for example

- algebraic manipulation: $x^2 - 2x + 3 = 0 \Rightarrow x = \frac{x^2 + 3}{2}$ or $x = \sqrt{2x - 3}$
- add x to both sides: $\sin x = 0 \Rightarrow x = x + \sin x$

- given an initial guess $x^{(1)}$, iterate

$$x^{(k+1)} = g(x^{(k)}), \quad k = 1, 2, \dots$$

called *fixed point iteration* (or *one-point iteration* or *successive substitution*)

- monitor the approximate relative error

$$\varepsilon_a = \left| \frac{x^{(k+1)} - x^{(k)}}{x^{(k+1)}} \right| \times 100\%$$

Example

$$f(x) = e^{-x} - x$$

true root: $x^{\star} \approx 0.567143140453502$

separate directly and iterate $x^{(k+1)} = g(x^{(k)}) = e^{-x^{(k)}}$ with $x^{(1)} = 0$

k	$x^{(k)}$	$x^{(k+1)} = g(x^{(k)})$	ε_a	ε_t (true relative error)	$\varepsilon_{t,k}/\varepsilon_{t,k-1}$
1	0.0000	1.0000		76.32%	
2	1.0000	0.3679	100%	35.13%	46.03%
3	0.3679	0.6922	171.83%	22.05%	62.76%
4	0.6922	0.5005	46.85%	11.76%	53.31%
5	0.5005	0.6062	38.31%	6.89%	58.65%
6	0.6062	0.5454	17.45%	3.83%	55.62%
7	0.5454	0.5796	11.16%	2.20%	57.34%
8	0.5796	0.5601	5.90%	1.24%	56.36%
9	0.5601	0.5711	3.48%	0.71%	56.91%
10	0.5711	0.5649	1.93%	0.40%	56.60%
11	0.5649	0.5684	1.11%	0.23%	56.78%

error proportional to that in previous iteration (about 60%); *linear convergence*

Example

choose alternative formulation $x = g(x) = -\ln(x)$ with starting point $x^{(1)} = 0.5$

k	$x^{(k)}$	$g(x^{(k)})$	ε_a	ε_t	$\varepsilon_{t,k}/\varepsilon_{t,k-1}$
1	0.5000	0.6931		11.84%	
2	0.6931	0.3665	27.87%	22.22%	1.8766
3	0.3665	1.0037	89.12%	35.38%	1.5923
4	1.0037	-0.0037	63.48%	76.98%	2.1760

diverges in this case

MATLAB implementation of our example

```
clear,clc,format compact
g=@(x) exp(-x);
x0 = 0;
[xr,ea,iter] = fixpt(g,x0,1e-6);
X = ['The root = ',num2str(xr),' (ea = ',num2str(ea),'% in ',...
num2str(iter),' iterations)'];
disp(X)
```

with function

```
function [x1,ea,iter] = fixpt(g,x0,es,maxit)
iter = 0; ea = 100;
while (1)
x1 = g(x0);
iter = iter + 1;
if x1 ~= 0, ea = abs((x1 - x0)/x1)*100; end
if (ea <= es || iter >= maxit), break, end
x0 = x1;
end
end
```

running code gives

```
The root = 0.56714 (ea = 7.6775e-07% in 35 iterations)
```

Convergence via two-curve method

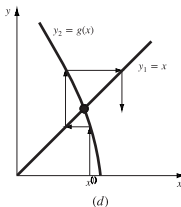
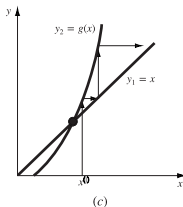
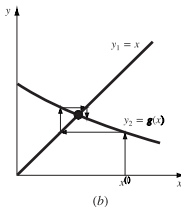
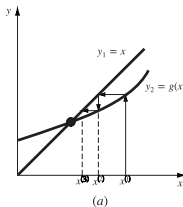
the two-curve method can be used to illustrate convergence and divergence

- equation $x = g(x)$ is re-expressed as a pair of equations: $y_1 = x$, $y_2 = g(x)$
- function $y_1 = x$ and $y_2 = g(x)$ are plotted
- roots of $f(x) = 0$ correspond to the abscissa at the intersection of the two curves

Iteration procedure

- start with initial guess $x^{(1)}$
- determine the corresponding point on y_2 curve: $(x^{(1)}, g(x^{(1)}))$
- move horizontally to y_1 : the point $(x^{(2)}, x^{(2)})$
- this corresponds to the first fixed-point iteration: $x^{(2)} = g(x^{(1)})$
- repeat starting from $x^{(2)}$ (moving to $(x^{(2)}, g(x^{(2)}))$ and then to $(x^{(3)}, x^{(3)})$)

Convergence via two-curve method



- (a) and (b): solution converges — estimates move closer to the root
- (c) and (d): iterations diverge away from the root

Convergence

fixed-point iteration converges locally (in a region of interest) if

$$|g'(x)| < 1$$

meaning the slope of $y_2 = g(x)$ is less than the slope of $y_1 = x$

- $0 < g'(x^\star) < 1$: monotone convergence
- $-1 < g'(x^\star) < 0$: oscillatory (spiral) convergence
- $|g'(x^\star)| > 1$: divergence

Proof: mean-value theorem with $x^{(k+1)} = g(x^{(k)})$, $x^\star = g(x^\star)$:

$$x^\star - x^{(k+1)} = g(x^\star) - g(x^{(k)}) = g'(\xi) (x^\star - x^{(k)})$$

thus for the true error $E_k = x^\star - x^{(k)}$,

$$E_{k+1} = g'(\xi) E_k \implies \text{if } |g'(\xi)| < 1 \text{ then } |E_{k+1}| < |E_k|$$

if converges, error decreases proportional to each step (linear convergence)

Fixed-point iteration for nonlinear systems

- fixed-point iteration extends to systems of nonlinear equations
- we describe the idea for two equations in two variables:

$$f_1(x_1, x_2) = 0, \quad f_2(x_1, x_2) = 0$$

Fixed point iteration

$$x_1^{(k+1)} = g_1(x_1^{(k)}, x_2^{(k)})$$

$$x_2^{(k+1)} = g_2(x_1^{(k)}, x_2^{(k)})$$

- also called *successive substitution*
- can also be applied sequentially:

$$x_1^{(k+1)} = g_1(x_1^{(k)}, x_2^{(k)})$$

$$x_2^{(k+1)} = g_2(x_1^{(k+1)}, x_2^{(k)})$$

where the variable x_2 uses most recent x_1

Example

$$f_1(x_1, x_2) = x_1^2 + x_1x_2 - 10 = 0, \quad f_2(x_1, x_2) = x_2 + 3x_1x_2^2 - 57 = 0$$

with $x_1^{(1)} = 1.5$ and $x_2^{(1)} = 3.5$

- formulate as

$$x_1^{(k+1)} = \frac{10 - (x_1^{(k)})^2}{x_2^{(k)}}, \quad x_2^{(k+1)} = 57 - 3x_1^{(k+1)}(x_2^{(k)})^2$$

- on the basis of the initial guesses, we have

$$x_1 = \frac{10 - (1.5)^2}{3.5} = 2.21429, \quad x_2 = 57 - 3(2.21429)(3.5)^2 = -24.37516$$

- one more iteration gives

$$x_1 = -0.20910, \quad x_2 = 429.709$$

Observation: the approach is diverging rapidly

Example

Alternative formulation: rearrange the equations as

$$x_1 = \sqrt{10 - x_1 x_2}, \quad x_2 = \sqrt{\frac{57 - x_2}{3x_1}}$$

- using initial guesses $x_1 = 1.5, x_2 = 3.5$:

$$x_1 = \sqrt{10 - 1.5(3.5)} = 2.17945$$

$$x_2 = \sqrt{\frac{57 - 3.5}{3(2.17945)}} = 2.86051$$

- next iteration:

$$x_1 = \sqrt{10 - 2.17945(2.86051)} = 1.94053$$

$$x_2 = \sqrt{\frac{57 - 2.86051}{3(1.94053)}} = 3.04955$$

Conclusion: the reformulated system converges to the true solution $x_1^* = 2, x_2^* = 3$

Remarks on convergence

- convergence depends heavily on formulation
- poor initial guesses can cause divergence
- sufficient (but restrictive) conditions:

$$\left| \frac{\partial f_1}{\partial x_1} \right| + \left| \frac{\partial f_1}{\partial x_2} \right| < 1, \quad \left| \frac{\partial f_2}{\partial x_1} \right| + \left| \frac{\partial f_2}{\partial x_2} \right| < 1$$

- limited utility for nonlinear systems, but useful for linear systems

Outline

- nonlinear equations
- fixed point iteration
- **Newton method for nonlinear equations**
- unconstrained optimization
- gradient and Newton methods for optimization

Deriving Newton method

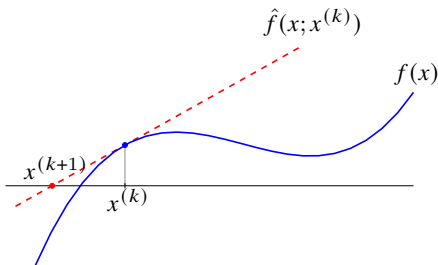
- linearize $f : \mathbb{R}^n \rightarrow \mathbb{R}^n$ (Taylor affine approximation) around current iterate $x^{(k)}$:

$$\hat{f}(x; x^{(k)}) = f(x^{(k)}) + Df(x^{(k)})(x - x^{(k)})$$

$Df(x)$ is the derivative (Jacobian) matrix of f

- take solution x of linearized equation $\hat{f}(x; x^{(k)}) = 0$ as the next iterate $x^{(k+1)}$:

$$x = x^{(k)} - Df(x^{(k)})^{-1} f(x^{(k)})$$



Newton method for nonlinear equations

assume $f : \mathbb{R}^n \rightarrow \mathbb{R}^n$ is differentiable

given a starting point $x^{(1)}$ and solution tolerance ϵ

repeat for $k \geq 1$

1. evaluate $Df(x^{(k)})$

2. set

$$x^{(k+1)} = x^{(k)} - Df(x^{(k)})^{-1} f(x^{(k)})$$

if $\|f(x^{(k+1)})\| < \epsilon$ (or $\|x^{(k+1)} - x^{(k)}\| < \epsilon$), stop and output $x^{(k+1)}$

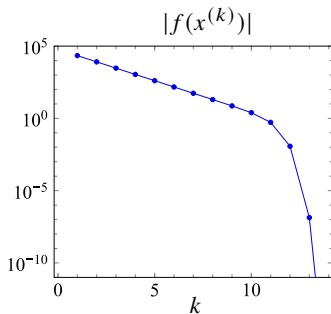
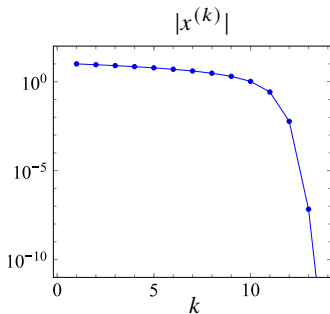
- derivative (Jacobian) matrix $Df(x^{(k)})$ is assumed to be nonsingular
- each iteration requires one evaluation of $f(x)$ and $Df(x)$
- each iteration requires factorization of the $n \times n$ matrix $Df(x)$
- also called *Newton-Raphson algorithm*

Example

Newton method applied to $f(x) = 2 \sinh(x) = e^x - e^{-x}$

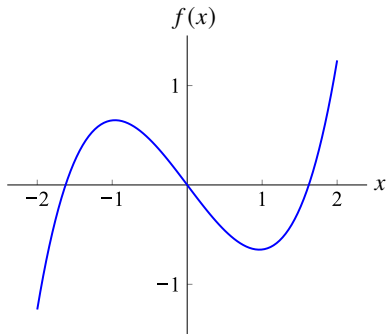
$$x^{(k+1)} = x^{(k)} - \frac{e^{x^{(k)}} - e^{-x^{(k)}}}{e^{x^{(k)}} + e^{-x^{(k)}}}$$

results with $x^{(1)} = 10$



Example

$$f(x) = e^x - e^{-x} - 3x$$

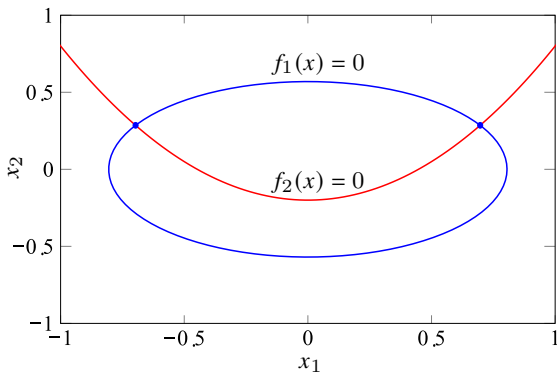


- starting point $x^{(1)} = -1$: converges to $x^{\star} = -1.62$
- starting point $x^{(1)} = -0.8$: converges to $x^{\star} = 1.62$
- starting point $x^{(1)} = -0.7$: converges to $x^{\star} = 0$

Example

$$f_1(x_1, x_2) = \log(x_1^2 + 2x_2^2 + 1) - 0.5 = 0$$

$$f_2(x_1, x_2) = x_2 - x_1^2 + 0.2 = 0$$



two equations in two variables; two solutions $(0.70, 0.29)$, $(-0.70, 0.29)$

Newton iteration

- evaluate $g = f(x)$ and

$$H = Df(x) = \begin{bmatrix} 2x_1/(x_1^2 + 2x_2^2 + 1) & 4x_2/(x_1^2 + 2x_2^2 + 1) \\ -2x_1 & 1 \end{bmatrix}$$

- solve $Hv = -g$ (two linear equations in two variables)
- update $x := x + v$

Results

- $x^{(1)} = (1, 1)$: converges to $x^* = (0.70, 0.29)$ in about 4 iterations
- $x^{(1)} = (-1, 1)$: converges to $x^* = (-0.70, 0.29)$ in about 4 iterations
- $x^{(1)} = (1, -1)$ or $x^{(1)} = (-1, -1)$: does not converge

Observations

- Newton's method works well if started near a solution; may not work otherwise
- can converge to different solutions depending on the starting point
- does not necessarily find the solution closest to the starting point

Convergence of Newton's method

if $f(x^\star) = 0$ and $Df(x^\star)$ is nonsingular, and $x^{(1)}$ is sufficiently close to $x^\star$, then

$$x^{(k)} \rightarrow x^\star, \quad \|x^{(k+1)} - x^\star\| \leq c \|x^{(k)} - x^\star\|^2$$

for some $c > 0$

- has quadratic convergence when started near a solution
- explains fast convergence when started near solution

Outline

- nonlinear equations
- fixed point iteration
- Newton method for nonlinear equations
- **unconstrained optimization**
- gradient and Newton methods for optimization

Hessian

Hessian of $g : \mathbb{R}^n \rightarrow \mathbb{R}$: a symmetric $n \times n$ matrix

$$\nabla^2 g(x) = \begin{bmatrix} \frac{\partial^2 g}{\partial x_1^2}(x) & \frac{\partial^2 g}{\partial x_1 \partial x_2}(x) & \cdots & \frac{\partial^2 g}{\partial x_1 \partial x_n}(x) \\ \frac{\partial^2 g}{\partial x_2 \partial x_1}(x) & \frac{\partial^2 g}{\partial x_2^2}(x) & \cdots & \frac{\partial^2 g}{\partial x_2 \partial x_n}(x) \\ \vdots & \vdots & \cdots & \vdots \\ \frac{\partial^2 g}{\partial x_n \partial x_1}(x) & \frac{\partial^2 g}{\partial x_n \partial x_2}(x) & \cdots & \frac{\partial^2 g}{\partial x_n^2}(x) \end{bmatrix}$$

- $\nabla^2 g(x)_{ij} = \frac{\partial^2 g}{\partial x_i \partial x_j}(x)$
- this is also the derivative matrix of gradient $\nabla g(x)$ at z

Quadratic (second order) approximation of g around z

$$g_q(x) = g(z) + \nabla g(z)^T(x - z) + \frac{1}{2}(x - z)^T \nabla^2 g(z)(x - z)$$

when $n = 1$ this reduces to

$$g_q(x) = g(z) + g'(z)(x - z) + \frac{1}{2}g''(z)(x - z)^2$$

Examples

Affine function: $g(x) = a^T x + b$

$$\nabla g(x) = a, \quad \nabla^2 g(x) = 0$$

Quadratic function: $g(x) = x^T P x + q^T x + r$ with P symmetric

$$\nabla g(x) = 2Px + q, \quad \nabla^2 g(x) = 2P$$

Least squares cost: $g(x) = \|Ax - b\|^2 = x^T A^T A x - 2b^T A x + b^T b$

$$\nabla g(x) = 2A^T A x - 2A^T b, \quad \nabla^2 g(x) = 2A^T A$$

Composition with affine mapping: if $g(x) = h(Cx + d)$, then

$$\begin{aligned}\nabla g(x) &= C^T \nabla h(Cx + d) \\ \nabla^2 g(x) &= C^T \nabla^2 h(Cx + d) C\end{aligned}$$

Example

$$g(x_1, x_2) = e^{x_1+x_2-1} + e^{x_1-x_2-1} + e^{-x_1-1}$$

gradient is

$$\nabla g(x) = \begin{bmatrix} e^{x_1+x_2-1} + e^{x_1-x_2-1} - e^{-x_1-1} \\ e^{x_1+x_2-1} - e^{x_1-x_2-1} \end{bmatrix}$$

Hessian is

$$\nabla^2 g(x) = \begin{bmatrix} e^{x_1+x_2-1} + e^{x_1-x_2-1} + e^{-x_1-1} & e^{x_1+x_2-1} - e^{x_1-x_2-1} \\ e^{x_1+x_2-1} - e^{x_1-x_2-1} & e^{x_1+x_2-1} + e^{x_1-x_2-1} \end{bmatrix}$$

using composition property we can express g as $g(x) = h(Cx + d)$ with

$$h(y_1, y_2, y_3) = e^{y_1} + e^{y_2} + e^{y_3}, \quad C = \begin{bmatrix} 1 & 1 \\ 1 & -1 \\ -1 & 0 \end{bmatrix}, \quad d = \begin{bmatrix} -1 \\ -1 \\ -1 \end{bmatrix}$$

Gradient: $\nabla g(x) = C^T \nabla h(Cx + d)$

$$\nabla g(x) = \begin{bmatrix} 1 & 1 & -1 \\ 1 & -1 & 0 \end{bmatrix} \begin{bmatrix} e^{x_1+x_2-1} \\ e^{x_1-x_2-1} \\ e^{-x_1-1} \end{bmatrix}$$

Hessian: $\nabla^2 g(x) = C^T \nabla^2 h(Cx + d) C$

$$\nabla^2 g(x) = \begin{bmatrix} 1 & 1 & -1 \\ 1 & -1 & 0 \end{bmatrix} \begin{bmatrix} e^{x_1+x_2-1} & 0 & 0 \\ 0 & e^{x_1-x_2-1} & 0 \\ 0 & 0 & e^{-x_1-1} \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & -1 \\ -1 & 0 \end{bmatrix}$$

Unconstrained minimization problem

$$\text{minimize } g(x_1, x_2, \dots, x_n)$$

- $g : \mathbb{R}^n \rightarrow \mathbb{R}$ is the *cost* or *objective* function
- $x = (x_1, x_2, \dots, x_n)$ is n -vector of optimization *variables*
- to solve a maximization problem (*i.e.*, maximize $g(x)$), we can minimize $-g(x)$
- we will assume that g is twice differentiable

Local and global optimum

- $x^\star$ is an **optimal point** (or a **minimum**) if

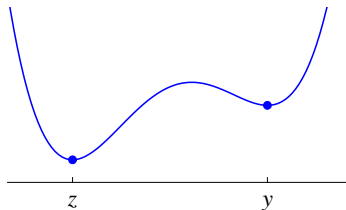
$$g(x^\star) \leq g(x) \quad \text{for all } x$$

also called globally optimal

- $x^\star$ is a **locally optimal point** (**local minimum**) if for some $R > 0$

$$g(x^\star) \leq g(x) \quad \text{for all } x \text{ with } \|x - x^\star\| \leq R$$

Example



z is (globally) optimal

y is locally optimal

First-order optimality conditions

a necessary condition for a minimum is

$$\nabla g(x^\star) = 0$$

a point where the gradient vanishes $\nabla g(x) = 0$ is called a *critical* or *stationary* point

- if $x^\star$ is a local minimum, then for any direction v we have

$$g(x^\star + v) = g(x^\star) + \nabla g(x^\star)^T v + (1/2)v^T \nabla^2 g(x^\star) v \geq g(x^\star)$$

- for a very small $\|v\|$, if $\nabla g(x^\star) \neq 0$, then we can find v such that $\nabla g(x^\star)^T v < 0$
- so we must have $\nabla g(x^\star) = 0$ at a minimum and $\nabla^2 g(x^\star)$ positive semidefinite
- gradient also vanish at maximum and saddle points (not minimum or maximum)
- at a strict minimum we must also have for all v satisfying $0 < \|v\| \ll 1$

$$g(x^\star + v) = g(x^\star) + (1/2)v^T \nabla^2 g(x^\star) v > g(x^\star)$$

this implies the Hessian matrix $\nabla^2 g(x^\star)$ is positive definite

Optimality conditions

Necessary condition: if $x^\star$ is locally optimal, then

$$\nabla g(x^\star) = 0 \quad \text{and} \quad \nabla^2 g(x^\star) \text{ is positive semidefinite}$$

Sufficient condition: if $x^\star$ satisfies

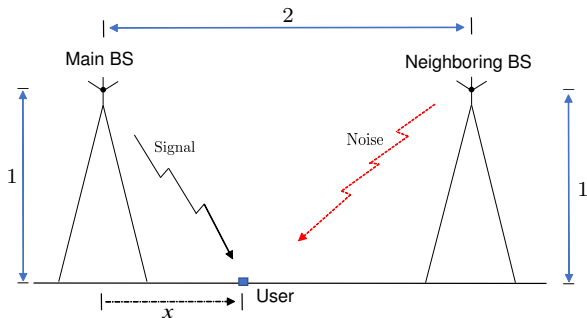
$$\nabla g(x^\star) = 0 \quad \text{and} \quad \nabla^2 g(x^\star) \text{ is positive definite}$$

then $x^\star$ is locally optimal

Necessary and sufficient condition for convex functions

- g is called *convex* if $\nabla^2 g(x)$ is positive semidefinite everywhere
- if g is convex then $x^\star$ is optimal if and only if $\nabla g(x^\star) = 0$

Example



- power of the received signal measured by the user from each antenna is the reciprocal of the squared distance from the corresponding antenna
- find position x of user (relative to main station) that maximizes signal-to-noise ratio

to solve this problem, we need to maximize the signal-to-noise ratio:

$$g(x) = \frac{1 + (2 - x)^2}{1 + x^2}$$

setting the derivative to zero:

$$g'(x) = \frac{-2(2 - x)(1 + x^2) - 2x(1 + (2 - x)^2)}{(1 + x^2)^2} = \frac{4(x^2 - 2x - 1)}{(1 + x^2)^2} = 0$$

- $g'(x) = 0$ at $x = 1 \pm \sqrt{2}$
- checking the objective values, we see that $x = 1 - \sqrt{2}$ gives larger objective
- derivative changes sign $+$ to $-$ when passing through $x = 1 - \sqrt{2}$, so $f''(x) < 0$
- hence, $x^\circ = 1 - \sqrt{2}$ is a local maximizer
- it is a global maximizer since $g(x) \rightarrow 1 < g(x^\circ)$ as $|x| \rightarrow \infty$

Examples ($n = 1$)

- $g(x) = \log(e^x + e^{-x})$

$$g'(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}, \quad g''(x) = \frac{4}{(e^x + e^{-x})^2}$$

$g''(x) \geq 0$ everywhere; $x^\star = 0$ is the unique optimal point

- $g(x) = x^4$

$$g'(x) = 4x^3, \quad g''(x) = 12x^2$$

$g''(x) \geq 0$ everywhere; $x^\star = 0$ is the unique optimal point

- $g(x) = x^3$

$$g'(x) = 3x^2, \quad g''(x) = 6x$$

$g'(0) = 0, g''(0) = 0$ but $x = 0$ is not locally optimal

Examples

- $g(x) = x^T Px + q^T x + r$ (P is symmetric positive definite)

$$\nabla g(x) = 2Px + q, \quad \nabla^2 g(x) = 2P$$

$\nabla^2 g(x)$ is positive definite everywhere, hence the unique optimal point is

$$x^\star = -(1/2)P^{-1}q$$

- $g(x) = \|Ax - b\|^2$ (A is a matrix with linearly independent columns)

$$\nabla g(x) = 2A^T Ax - 2A^T b, \quad \nabla^2 g(x) = 2A^T A$$

$\nabla^2 g(x)$ is positive definite everywhere, hence the unique optimal point is

$$x^\star = (A^T A)^{-1} A^T b$$

Examples

$$g(x_1, x_2) = e^{x_1+x_2-1} + e^{x_1-x_2-1} + e^{-x_1-1}$$

- we can express $\nabla^2 g(x)$ as

$$\nabla^2 g(x) = \begin{bmatrix} 1 & 1 & 1 \\ 1 & -1 & 0 \end{bmatrix} \begin{bmatrix} e^{x_1+x_2-1} & 0 & 0 \\ 0 & e^{x_1-x_2-1} & 0 \\ 0 & 0 & e^{-x_1-1} \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & -1 \\ 1 & 0 \end{bmatrix}$$

this shows that $\nabla^2 g(x)$ is positive definite for all x

- therefore $x^\star$ is optimal if and only if

$$\nabla g(x^\star) = \begin{bmatrix} e^{x_1^\star+x_2^\star-1} + e^{x_1^\star-x_2^\star-1} - e^{-x_1^\star-1} \\ e^{x_1^\star+x_2^\star-1} - e^{x_1^\star-x_2^\star-1} \end{bmatrix} = 0$$

two nonlinear equations in two variables

Outline

- nonlinear equations
- fixed point iteration
- Newton method for nonlinear equations
- unconstrained optimization
- **gradient and Newton methods for optimization**

Descent direction

Descent direction: a vector $v \in \mathbb{R}^n$ is called a *descent direction* for g if

$$g(x + tv) < g(x) \quad \text{for sufficiently small } t > 0$$

Directional derivative

- for given z and nonzero v , define $h(t) = g(z + tv)$
- derivative of h at $t = 0$

$$\begin{aligned} h'(0) &= \frac{\partial g}{\partial x_1}(z)v_1 + \frac{\partial g}{\partial x_2}(z)v_2 + \cdots + \frac{\partial g}{\partial x_n}(z)v_n \\ &= \nabla g(z)^T v \end{aligned}$$

is called the *directional derivative* of g (at z , in the direction v)

- $\nabla g(x)^T v$ gives an approximate rate of increase of g in the direction v at x
- a vector $v \in \mathbb{R}^n$ is a descent direction if $\nabla g(x)^T v < 0$

Gradient descent method

the directional derivative of f at x in the direction $v = -\nabla g(x)$ is

$$v^T \nabla g(x) = -\|\nabla g(x)\|^2 < 0$$

for any x with $\nabla g(x) \neq 0$; thus, $-\nabla g(x)$ is a descent direction

Gradient descent algorithm

given a starting point $x^{(1)}$ and a solution tolerance $\epsilon > 0$

repeat for $k \geq 1$

1. choose a stepsize t_k
2. update

$$x^{(k+1)} = x^{(k)} - t_k \nabla g(x^{(k)})$$

if $\|\nabla f(x^{(k+1)})\| \leq \epsilon$ stop and output $x^{(k+1)}$

- t_k is called the *stepsize* or *learning rate*
- for t_k small enough, the algorithm is a descent method
- for large t_k is large enough, algorithm may not be a descent method and may fail

Determining the stepsize

suppose $v^{(k)}$ is any descent direction

Constant stepsize: set $t_k = t$ for all k

Backtracking line search

- choose $\beta \in (0, 1/2)$, $\gamma \in (0, 1)$ and an initial step size t_1 (e.g., $t_1 = 1$)
- set $t_k := \beta t_k$ until

$$g(x^{(k)} + t_k v^{(k)}) < g(x^{(k)}) + \gamma t_k \nabla g(x^{(k)})^T v^{(k)}$$

- simple backtracking algorithm is to set

$$t = 1, 0.5, 0.5^2, 0.5^3, \dots$$

until the above is satisfied or until $g(x^{(k)} + t_k v^{(k)}) < g(x^{(k)})$

Example

$$g(x_1, x_2, x_3) = (x_1 - 4)^4 + (x_2 - 3)^2 + 4(x_3 + 5)^4$$

the gradient of this function is

$$\nabla g(x) = \begin{bmatrix} 4(x_1 - 4)^3 \\ 2(x_2 - 3) \\ 16(x_3 + 5)^3 \end{bmatrix}$$

applying one iteration of gradient descent with $x^{(1)} = (4, 2, -1)$ and $t = 0.002$ gives

$$x^{(2)} = \begin{bmatrix} 4 \\ 2 \\ -1 \end{bmatrix} - 0.002 \begin{bmatrix} 4(4 - 4)^3 \\ 2(2 - 3) \\ 16(-1 + 5)^3 \end{bmatrix} = \begin{bmatrix} 4.000 \\ 2.004 \\ -3.048 \end{bmatrix}$$

notice that

$$59.06 = g(4, 2.004, -3.048) < g(4, 2, -1) = 1025$$

this shows that $t = 0.002$ is a good choice

Newton method for minimizing a convex function

if $\nabla^2 g(x)$ is positive definite everywhere, we can minimize $g(x)$ by solving

$$\nabla g(x) = 0$$

Algorithm: choose $x^{(1)}$ and repeat for $k = 1, 2, \dots$

$$x^{(k+1)} = x^{(k)} - \nabla^2 g(x^{(k)})^{-1} \nabla g(x^{(k)})$$

- $v = -\nabla^2 g(x)^{-1} \nabla g(x)$ is called the *Newton step* at x , which is a descent direction
- converges if started sufficiently close to the solution
- Newton step is computed by a Cholesky factorization of the Hessian
- for $n = 1$, the iteration can be written as

$$x^{(k+1)} = x^{(k)} - \frac{g'(x^{(k)})}{g''(x^{(k)})}$$

Interpretations of Newton step

Affine approximation of gradient

- affine approximation of $f(x) = \nabla g(x)$ around $x^{(k)}$ is

$$\hat{f}(x; x^{(k)}) = \nabla g(x^{(k)}) + \nabla^2 g(x^{(k)})(x - x^{(k)})$$

- Newton update $x^{(k+1)}$ is solution of linear equation $\hat{f}(x; x^{(k)}) = 0$

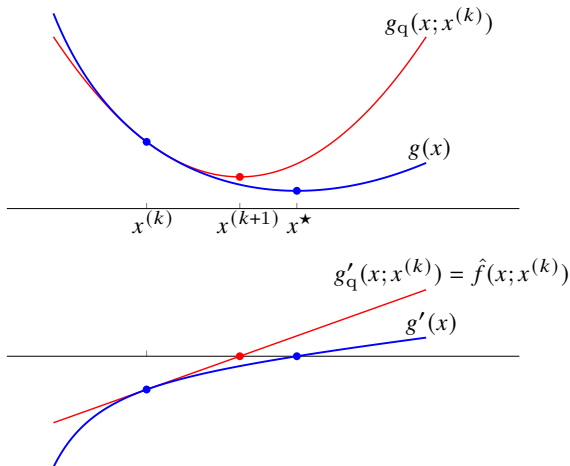
Quadratic approximation of function

- quadratic approximation of $g(x)$ around $x^{(k)}$ is

$$g_q(x; x^{(k)}) = g(x^{(k)}) + \nabla g(x^{(k)})^T (x - x^{(k)}) + \frac{1}{2} (x - x^{(k)})^T \nabla^2 g(x^{(k)}) (x - x^{(k)})$$

- Newton update $x^{(k+1)}$ satisfies $\nabla g_q(x; x^{(k)}) = 0$

Example ($n = 1$)



$$g_q(x; x^{(k)}) = g(x^{(k)}) + g'(x^{(k)})(x - x^{(k)}) + \frac{g''(x^{(k)})}{2}(x - x^{(k)})^2$$

Example

$$\text{minimize } g(x) = \frac{1}{2}x^2 - \sin x$$

- applying Newton's method with $x^{(1)} = 0.5$, we have

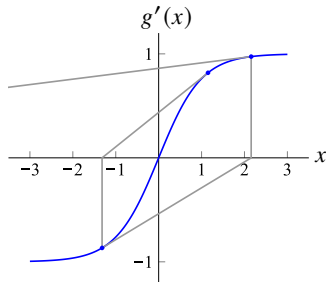
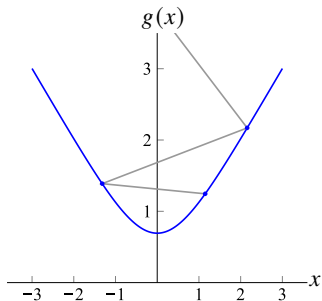
$$\begin{aligned}x^{(2)} &= x^{(1)} - \frac{g'(x^{(1)})}{g''(x^{(1)})} = 0.5 - \frac{0.5 - \cos(0.5)}{1 + \sin(0.5)} \\&= 0.5 - \frac{-0.3775}{1.479} = 0.7552\end{aligned}$$

repeating, we get $x^{(3)} = 0.7391$, $x^{(4)} = 0.7390$, and $x^{(5)} \approx 0.7390$

- note that $g'(x^{(5)}) \approx 0$, and $g''(x^{(5)}) = 1.672 > 0$
- so, $x^{(5)}$ is an approximate local minimizer (it is an approximate global minimizer)

Example

$$g(x) = \log(e^x + e^{-x}), \quad g'(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}, \quad g''(x) = \frac{4}{(e^x + e^{-x})^2}$$



does not converge when started at $x^{(1)} = 1.15$

Damped Newton method

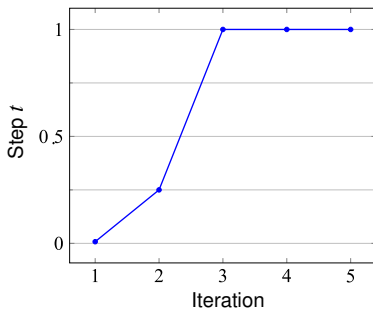
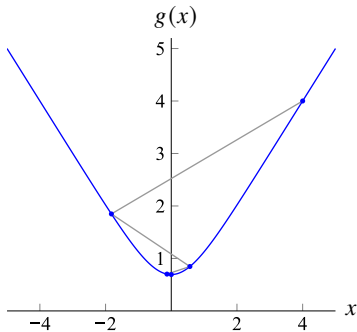
given a starting point $x^{(1)}$

repeat for $k = 1, 2, \dots$

1. compute Newton step $v = -\nabla^2 g(x^{(k)})^{-1} \nabla g(x^{(k)})$
 2. select a stepsize t (e.g., using backtracking line search)
 3. update $x^{(k+1)} = x^{(k)} + tv$
-

Example

$$g(x) = \log(e^x + e^{-x}), \quad x^{(1)} = 4$$

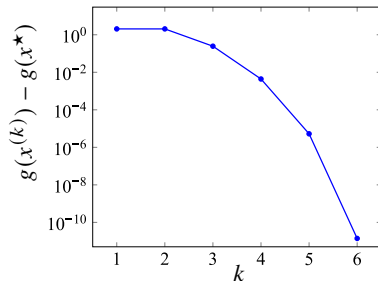
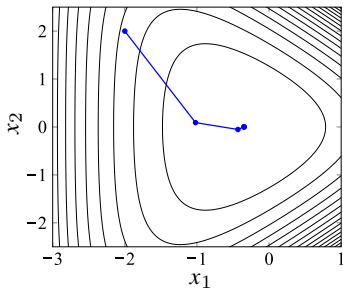


close to the solution: very fast convergence, no backtracking steps

Example

$$g(x_1, x_2) = e^{x_1+x_2-1} + e^{x_1-x_2-1} + e^{-x_1-1}$$

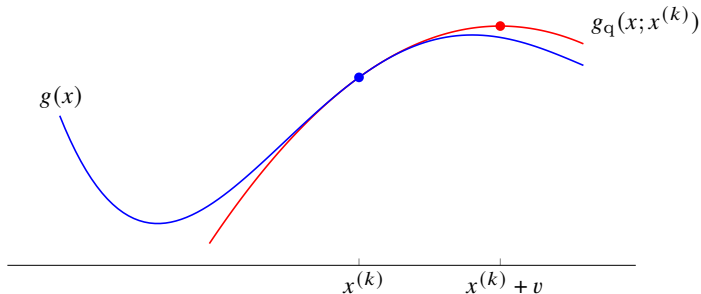
damped Newton method started at $x = (-2, 2)$



Newton method for nonconvex functions

if $\nabla^2 g(x^{(k)})$ is not positive definite, it is possible that Newton step v satisfies

$$\nabla g(x^{(k)})^T v = -\nabla g(x^{(k)})^T \nabla^2 g(x^{(k)})^{-1} \nabla g(x^{(k)}) > 0$$



- if Newton step is not descent direction, replace it with descent direction
- simplest choice is $v = -\nabla g(x^{(k)})$ or $v = -(\nabla^2 g(x_k) + \mu_k I)^{-1} \nabla g(x^{(k)})$

References and further readings

- S. Boyd and L. Vandenberghe. *Introduction to Applied Linear Algebra: Vectors, Matrices, and Least Squares*. Cambridge University Press, 2018.
- L. Vandenberghe, *EE133A Lecture Notes*, University of California, Los Angeles.